Week 1: MLFQ Scheduler Design Document

Multi-Level Feedback Queue Scheduler for xv6-RISC-V

Abdullah Khan Sherwani 27880
Muhammad Anas Tabba 28998

3rd December 2025

Contents

1 Introduction

This document describes the design and foundation phase for implementing a Multi-Level Feedback Queue (MLFQ) scheduler in xv6-RISC-V. The MLFQ scheduler replaces xv6's default round-robin scheduler with a priority-based scheduler that dynamically adjusts process priorities based on their CPU usage behavior.

1.1 Project Objectives

- Replace xv6's simple round-robin scheduler with MLFQ
- Implement 4 priority queues with different time quanta
- Automatically demote CPU-bound processes to lower priorities
- Favor I/O-bound processes for better interactive responsiveness
- Prevent starvation through periodic priority boosting

1.2 Week 1 Goals

1. Design the MLFQ queue structure and scheduling policies
2. Add necessary data structures to the process control block
3. Implement `getprocinfo` system call for debugging
4. Create foundation for time slice tracking
5. Prepare test infrastructure

2 MLFQ Design Specifications

2.1 Queue Architecture

The MLFQ scheduler uses 4 priority queues:

Queue	Priority	Time Quantum	Description
Q0	Highest	2 ticks (200ms)	Interactive/I/O-bound processes
Q1	High	4 ticks (400ms)	Medium-priority processes
Q2	Medium	8 ticks (800ms)	Compute-intensive processes
Q3	Lowest	16 ticks (1.6s)	Long-running CPU-bound processes

Table 1: MLFQ Queue Configuration

2.2 Scheduling Policy

1. **Initial Placement:** All new processes start at Q0 (highest priority)
2. **Round-Robin per Queue:** Within each queue, processes are scheduled in round-robin order
3. **Priority Selection:** Scheduler always picks from the highest non-empty queue
4. **Demotion:** Process that uses its full time quantum is demoted to the next lower queue
5. **Voluntary Yield:** Process that yields before its quantum expires keeps its priority

2.3 Starvation Prevention

To prevent lower-priority processes from starving:

- **Priority Boost:** Every 30 ticks, all processes are boosted back to Q0
- This ensures even CPU-bound processes periodically get high-priority treatment

3 Implementation Details

3.1 Process Structure Modifications

Added MLFQ-specific fields to `struct proc` in `kernel/proc.h`:

```
// Per-process state
struct proc {
  struct spinlock lock;
  enum procstate state;        // Process state
  void *chan;                  // If non-zero, sleeping on chan
  int killed;                  // If non-zero, have been killed
  int xstate;                  // Exit status
  int pid;                     // Process ID

  // ... existing fields ...

  // MLFQ scheduler fields
  int priority;                // Current priority queue (0-3, 0
   is highest)
  int time_slices;             // Time slices used in current
   priority level
  uint64 arrival_time;         // Time when process entered
   current queue
};
```

Listing 1: MLFQ Fields in struct proc (kernel/proc.h)

3.2 Global MLFQ Configuration

Added global MLFQ data structures in `kernel/proc.c`:

```
// MLFQ scheduler data structures
#define NMLFQ 4  // Number of priority queues (0=highest, 3=lowest)

// Time slices per queue - adjusted for xv6's timer tick rate (~100ms/tick)
// Q0: 2 ticks, Q1: 4 ticks, Q2: 8 ticks, Q3: 16 ticks
#define BOOST_INTERVAL 30  // Boost all processes every 30 ticks

uint64 last_boost_time = 0;
int mlfq_time_quanta[NMLFQ] = {2, 4, 8, 16};
struct spinlock mlfq_lock;
```

Listing 2: MLFQ Configuration (kernel/proc.c)

3.3 Process Initialization

Modified `allocproc()` to initialize MLFQ fields:

```
static struct proc*
allocproc(void)
{
  struct proc *p;
  // ... existing allocation code ...

found:
  p->pid = allocpid();
  p->state = USED;

  // ... trapframe and pagetable setup ...

  // Initialize MLFQ fields - new processes start at highest priority
  p->priority = 0;
  p->time_slices = 0;
  p->arrival_time = 0;

  return p;
}
```

Listing 3: Process MLFQ Initialization (kernel/proc.c)

3.4 System Call: getprocinfo

Implemented a system call to retrieve process scheduling information:

```
// Get process information for MLFQ debugging
uint64
sys_getprocinfo(void)
```

```
{
  uint64 addr;
  struct proc *p = myproc();

  argaddr(0, &addr);

  // Create a structure to hold the info to copy out
  struct {
    int pid;
    int state;
    int priority;
    int time_slices;
  } info;

  acquire(&p->lock);
  info.pid = p->pid;
  info.state = p->state;
  info.priority = p->priority;
  info.time_slices = p->time_slices;
  release(&p->lock);

  if(copyout(p->pagetable, addr, (char *)&info, sizeof(info)) <
   0)
    return -1;

  return 0;
}
```

Listing 4: getprocinfo System Call (kernel/sysproc.c)

3.5 User-Space Interface

Added to `user/user.h`:

```
// Structure for getprocinfo syscall
struct procinfo {
  int pid;            // Process ID
  int state;          // Process state
  int priority;       // Priority queue (0-3)
  int time_slices;    // Time slices used
};

// System call declarations
int getprocinfo(struct procinfo*);
int boostproc(void);
```

Listing 5: User-Space procinfo Structure (user/user.h)

4 Files Modified

4.1 Kernel Files

- `kernel/proc.h` – Added MLFQ fields to `struct proc`
- `kernel/proc.c` – Added MLFQ globals, initialization in `procinit()` and `allocproc()`
- `kernel/syscall.h` – Added `SYS_getprocinfo` (syscall 22)
- `kernel/syscall.c` – Registered `sys_getprocinfo` handler
- `kernel/sysproc.c` – Implemented `sys_getprocinfo()`

4.2 User Files

- `user/user.h` – Added `struct procinfo` and function declarations
- `user/usys.pl` – Added `getprocinfo` stub generator
- `user/procinfo.c` – Created test program

4.3 Build System

- `Makefile` – Set `CPUS := 1`, added test programs to `UPROGS`

5 Testing

5.1 Test Program: procinfo

Created a simple test to verify the `getprocinfo` system call:

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int main(int argc, char *argv[])
{
  struct procinfo info;

  printf("Process Information:\n");

  if(getprocinfo(&info) == 0) {
    printf("  PID: %d\n", info.pid);
    printf("  State: %d (RUNNING)\n", info.state);
    printf("  Priority Queue: Q%d\n", info.priority);
    printf("  Time Slices Used: %d\n", info.time_slices);
    printf("\nSUCCESS: getprocinfo syscall working!\n");
  } else {
    printf("FAILED: Could not get process info\n");
  }
```

```
20
21    exit(0);
22 }
```

Listing 6: procinfo Test Program (user/procinfo.c)

5.2 Expected Output

```
1 $ procinfo
2 Process Information:
3   PID: 3
4   State: 4 (RUNNING)
5   Priority Queue: Q0
6   Time Slices Used: 0
7
8 SUCCESS: getprocinfo syscall working!
```

Listing 7: Expected Terminal Output

6 Week 1 Summary

6.1 Completed Tasks

Designed 4-level MLFQ queue structure

Defined time quanta: 2, 4, 8, 16 ticks per queue

Added MLFQ fields to process control block

Implemented `getprocinfo` system call

Created test program to validate syscall

Configured single-CPU mode for simplicity

6.2 Next Steps (Week 2)

1. Implement actual MLFQ scheduling logic in `scheduler()`
2. Add time slice tracking in timer interrupt handler
3. Implement automatic demotion policy
4. Create CPU-bound and I/O-bound test programs

7 Appendix: Design Rationale

7.1 Why 4 Priority Levels?

- Provides enough granularity to differentiate process behavior
- Standard in MLFQ implementations (e.g., BSD scheduler)
- Minimal overhead for queue management

7.2 Why Exponential Time Quanta?

- Matches MLFQ literature and best practices
- Short slices for interactive processes (quick response)
- Long slices for CPU-bound processes (reduced context switches)

7.3 Why Single-Core Mode?

- Simplifies synchronization and debugging
- Focuses on the scheduling algorithm, not concurrency
- Easier to trace and verify scheduler behavior